

Extensible Math Functions for C++

Document #: P41884R1
Date: 2026-03-30
Project: Programming Language C++
Audience: SG6
SG18
SG20
LEWG
Reply-to: Stéphane Gros-Lemesre

<stephane.groslemesre@gmail.com>

Abstract

This paper proposes making standard library mathematical functions extensible to user-defined types via ADL. To preserve backward compatibility, the proposed approach introduces a new `std::math` sub-namespace. The new namespace also offers an opportunity to introduce a C++-idiomatic math library consistent with general standard library naming patterns.

Contents

1	Motivation	2
1.1	Problem	2
1.1.1	Naive call sites	2
1.1.2	The <code>using</code> -statement workaround	2
2	References	4

1 Motivation

1.1 Problem

User-defined types are central to C++ programming. The standard library inherited from the Standard Template Library its foundational principle of separating data structures and algorithms through generic programming. Containers, iterators, Customization Point Objects (CPOs), custom operators all contribute to enable a natural integration of any type, user-defined or not, in a common ecosystem.

Mathematical functions are a notable gap in this picture. Unlike operators, they are not part of overload resolution in a way that extends to user-defined types, and unlike CPOs, there is no standard hook to participate in. An author implementing an arithmetic-like type will find it easy to implement binary operations such as addition, subtraction, multiplication, and division; comparisons, hashing, or integration with containers for their user-defined type, until they face the problem of implementing square-root operation or similar mathematical function.

The fundamental problem with defining mathematical functions for a custom type is that because there are no well-defined contract like for the operators, and no CPO in the standard library for that purpose, the mechanism for their integration is not in the hands of the author. A custom implementation can be offered, but whether it will be picked up by generic code depends entirely on how that code was implemented.

This is not what good separation of concerns looks like: for the author of numeric types, providing custom mathematical functions is a leap of faith, while for generic code authors, supporting custom types might not be the primary concern and requires a significant amount of boilerplate.

1.1.1 Naive call sites

It is difficult to quantify the amount of generic code calling mathematical functions without support for user-defined numeric types. This is made even more difficult as these functions can live in the global namespace if `<math.h>` is included. Using GitHub search functionality shows 115k C++ files using the idiom `using std::pow;` for 3.5M C++ files calling `pow()`, of which 741k C++ files calling explicitly `std::pow()`.

Although these numbers have to be taken with caution, as there are other ways to leverage ADL for the `pow` function than `using std::pow;` specifically, and some of the qualified `std::pow` calls might come from non-generic code, it seems likely that a non-negligible amount of generic code does not allow custom types because of this extensibility limitation.

The library `nholthaus/units`, for instance, provides `units::math::sqrt` which calls `std::sqrt` directly on the underlying scalar value, which means it does not enable ADL resolution and thus does not extend to custom underlying types. [\[nholthaus-units\]](#)

This should not be surprising: the responsibility of making custom types work intuitively should belong to their authors, not to their users or third-party. It takes conscious effort for a generic library author to anticipate the needs for wrappers of hypothetical custom types and explicitly provide support for them.

Yet, the consequence of this status quo is substantial: the successful integration of a custom-type in a code-base no longer depends on its own design and implementation, but on the implementation of every other piece of code that will manipulate it. It is effectively a dependency of that type on all of the generic code it interacts and will interact with in the future. Such open-ended, long-term consequences are often unacceptable.

This is significant because custom numeric types can be used to support **correctness and safety** by encoding constraints that are enforced by the type system automatically. This pattern is similar to *newtypes* in Rust, and is sometimes referred to as “semantic-typing”. Whether it is by preventing inappropriate conversion or ensuring unit consistency, it can prevent entire classes of bugs and is a well-understood tool for writing safer numeric code. As it stands, its practicality in C++ is significantly restrained by how well such custom types compose with the broader ecosystem, and mathematical functions are a substantial weak point in that composition.

1.1.2 The using-statement workaround

The idiomatic workaround is to enable ADL via a `using` declaration:

```
using std::sqrt;
return sqrt(x);
```

This allows a user-defined mathematical function in the same namespace as the type passed into the function to be found via ADL, while falling back to the `std`-defined version for built-in types. However, this idiom has a critical limitation: it requires a statement, and is therefore unavailable in contexts that only accept expressions.

1.1.2.1 Statement-only

Constructor member initializer lists:

```
MyType(A aSq, B b, C c)
: a(sqrt(aSq)), // std::sqrt not found through conversion
  b(b),
  c(abs(c))      // std::abs not found through conversion
{}
```

The same applies to default member initializers:

```
struct Foo {
    double x = sqrt(v); // std::sqrt not found through conversion
};
```

requires expressions:

```
template<typename T>
concept numeric = requires(T x) {
    { abs(x) }; // requirement fails
};
```

There are other, less obvious situations where the same limitation applies, such as constant expressions (array size from a new-type) and template arguments.

In all of these cases, the programmer faces the same three unsatisfactory choices:

Option 1: Call `std::` explicitly

```
MyType(A aSq, B b, C c)
: a(std::sqrt(aSq)), // silently breaks extensibility
  b(b),
  c(std::abs(c))
{}
```

This compiles and works for built-in types, but silently cuts off any user-defined overload.

Option 2: Restructure to allow a statement

For member initializer lists, this means moving initialization to the constructor body:

```
MyType(A aSq, B b, C c)
: b(b)
{
    using namespace std;
    this->a = sqrt(aSq); // requires a to be default-constructible
    this->c = abs(c);     // loses const and reference member support
}
```

This restores ADL but members can no longer be `const` or references, and all members must be default-constructible. Not all contexts can be restructured this way.

Option 3: Write boilerplate helper functions

```

template<typename T> auto my_sqrt(T&& x) {
    using std::sqrt;
    return sqrt(std::forward<T>(x));
}

MyType(A aSq, B b, C c)
    : a(my_sqrt(aSq)),
      b(b),
      c(my_abs(c))
{}

```

This restores extensibility but requires every author of generic code to write and maintain their own dispatch layer; one wrapper per math function, 45+ at least, and likely more in the future (see [P3935R1]).

The ownership of these wrappers is unclear. Custom numeric-type authors should probably embed them in their math function implementations, but authors of generic code using such functions cannot rely on this being provided and may need to implement them redundantly to support a wider range of types.

2 References

- [nholthaus-units] Nick Holthaus. nholthaus/units Library.
<https://github.com/nholthaus/units>
- [P3935R1] Jan Schultke. 2026-05-11. Rebasing <cmath> on C23.
<https://wg21.link/p3935r1>